

Document number: P0680R0

Date: 2017-06-14

Author: Torvald Riegel triegel@redhat.com

Audience: SG1

SG1 efficiency

While we, meaning the people participating in SG1, do produce results continuously, I think we can improve the efficiency of how we do that; given that our resources are limited, efficiency improvements will likely also result in higher SG1 productivity. In particular, I think we can learn from what good open source projects do and what they pay attention to.

I'd like to keep this paper short, which may perhaps result in it reading like a rant. That's not the intention. But I think it is important and that we would really benefit from tackling the issues I'll describe next -- the sooner the better. While I suggest a few steps that should help us improve, this is really just a start. In the end, we need to own any change as a group anyway. By "own" I mean that we need to find consensus for any changes, and then strive to implement whatever we reached consensus on. Furthermore, every example I might make is just that, and not intended to blame somebody.

This started out as a discussion on the SG1 mailing list before the Kona meeting, but we didn't get to it at the meeting, and didn't continue to discuss after the meeting. This paper is an extension of that discussion. I was asked by a few people (outside of SG1) to write a paper so that it could get discussed; IMO, that kind of thinking is an example of what we need to change and improve :)

What characterizes us?

We are a community, not a tightly-coupled committee and a bunch of users on the other side. Roughly, I'd say that our community -- in the open-source project sense -- consists of:

- SG1 core: The core group of people in SG1 (ie, those who regularly attend meetings, present papers in person, etc.).
- SG1 contributors: people that are not in the core group but contribute directly (eg, who submit proposals but can't attend meetings).
- Followers: People that actively (try to) follow what SG1 does (e.g., people that teach others, early adopters, enthusiasts, ...).
- Users: People that use our stuff, including using TSeS. (I assume implementers are in one of the three previous groups.)

Even if just considering SG1 core (see below for more on reaching out to users), it's easy to see that this is not a homogeneous workforce who works for the same company and in the same building:

- We all have a day job, nobody works full-time on SG1 topics.
- Our individual schedules are not driven by a common set of deadlines or priorities (e.g., as if employed by one company), so participation in SG1 is not synchronized.
- We can't expect people to be able to participate all the time; before the meetings, we all try to get stuff done, but it's often not the best time for us individually.

Therefore, it is essential for us to make collaboration under these constraints possible and efficient. We depend on this collaboration; how efficient it is for people to contribute affects how many contributions we get. People will invest less in an inefficient process, or if there are obstacles to contribution.

By now, the similarity to what open-source projects face should be obvious. While such projects (e.g., the Linux Kernel project) build software and we are not tasked with building a reference implementation, there are a lot of similarities in how the work is conducted, and in what increases or decreases productivity. IOW, the key point is collaborating based on open source principles, not using an OSI-approved software licensed. Good open-source projects know how to use collaboration and diversity to their advantage, and we can learn from that.

What do we produce?

The obvious answer is that we produce specifications, so TSeS or IS changes. However, the other product we work on is the design of the programming abstractions we produce, which goes beyond the specifications: For example:

- The reasons why we design things in a certain way, trade-offs and assumptions that we make.
- How we think the abstractions should be used.
- What we have in mind for future revisions of the standard and how that is supposed to align with what is already specified.
- What SG1 as a group understands about the subject matter, things we are quite sure about vs. topics we disagree about, etc.

I think there sometimes is too much focus solely on the specifications. Of course, that's the tangible product and is what ISO cares about -- but the other things are equally important for both (1) our own process of creating specifications and (2) the specifications being most useful to the users we target.

The design is the core of what we do, and the specifications we produce are just the condensed form of this, covering a cut along the implementation/users interface. Unfortunately, we don't work on shareable representations of the design with the same level of focus as we do for the specifications. This is bad because to enable collaboration, the design, so the source from which we build stuff, needs to be easily shareable with other people. Doing so will allow newcomers to SG1 to get up to speed quicker, people can recover more quickly from missing a meeting or from forgetting something, and others outside of SG1 will have an easier time producing educational material. In contrast, if the design isn't easily shared, this will create friction in the collaboration process, such as cases when we're spending significant meeting time on trying to remember what we did at the last meeting, or when a paper coming out of SG1 isn't easy to assess by other people in the committee and thus voting is slowed down.

The importance of "sharing the source" is another example of why open source principles can be beneficial to us.

Things we can do to improve

I think we should start looking at three areas for improvement:

- Maintaining a shareable representation of SG1's "collective brain" so as to enable part-time participation. Participation should not be an all-or-nothing step function. Our focus should be on the total productivity of the community.
- Focusing on consensus building as the primary way of how we produce results (besides getting new proposals); whatever helps us with reaching consensus will help SG1 productivity.
- Avoiding that meetings become a bottleneck and enable/encourage continuous contributions and integration.

I believe none of my suggestions would conflict with the ISO rules. For example, everything we do between meetings could be required to be approved during a meeting; likewise, if we would set up additional rules with guidance for how paper authors' should address feedback, it would not mean that papers not following that advice are automatically rejected. I think that the key point is to show what improves SG1's efficiency, and that this will be sufficient encouragement for contributors to participate in the processes that make this more efficient.

Maintain a shareable representation of SG1's collective brain

By "collective brain", I primarily mean the design and the relevant things around it; not just a snapshot of some background for the current specifications, but why we designed something in a certain way, which design choices and assumptions we made, what we do and don't yet agree on, etc. So, IOW, the things a hypothetical person would know if that person would attend every meeting and conversation and wouldn't forget a thing. The level of detail should be so that it enables efficient collaboration with "the brain" without getting bogged down in the details.

To be "shareable", we need something written down for which changes can be tracked easily. While there are benefits to sharing this widely (see below), I think we should focus on just SG1 core and SG1 contributors for now (i.e., same scope as the wiki today).

The goal of this is enabling partial participation and making it more efficient. This matters because most of the people in SG1 are not able to follow each and every discussion in and between meetings, nor remembering everything in full detail. For example, one might need to be in CWG or another WG for a while during a meeting, or one might not remember a decision at a prior meeting or why that decision were made. The quicker people can catch up, the quicker can they contribute constructively to the design process, and the less overhead we will have in meetings.

We don't have such a representation right now:

- Meeting minutes are (supposed to be) a log of the conversations in the meeting, but they do not capture the change in the collective understanding of a topic reached during the meeting. Right now, it's very hard or impossible to catch up using minutes after missing a session or a full meeting.
- Some papers contain good discussions of the pros and cons, but a paper is typically an author's opinion; it is unlikely to cover all the other alternatives for solving the respective problem, or why those might have been rejected by SG1 or not.
- The lifetime of a paper naturally ends when it gets adopted or rejected. We cannot expect papers to continue being revised if the area they touched gets changed after the paper's adoption or rejection.

Here is what I would suggest for how to maintain such a representation:

- Per topic or paper, maintain the following information:
 - Open questions: Allows tracking of questions that we think we need to reach consensus on, or need further investigation on.
 - Established consensus: Points we agreed on as a group (i.e., we have consensus on them), so we can refer to them and know exactly what to do when we may to revisit a decision.
 - Action items: So we have a simple form of task tracking.
 - A brief design overview and how it relates to the overall design of all SG1 features.
 - All other information that may get lost otherwise but could be important for us in the future (unless that's in the paper already).
- Use text files with light markup, such as Markdown formatting. For example, one file per paper or topic.
- Use a git repository to track changes, along with one of the commonly used processes for patch review or merge requests. That's important because it allows people to catch up by looking at the commit log and the patches.

One comment that is sometimes made regarding doing something like this is that it would be further overhead. However, I think the overhead of not doing it is larger. First, we all have to do something like that, at the very least inside of our individual brains. Second, we don't do that perfectly, so we spend a significant amount of time, both individually and as a group at meetings, to recover parts of that information -- repeatedly. Third, we spend time explaining SG1's state and the state of proposals to others.

Another comment that is sometimes made is that we'd need a volunteer for that, and volunteers are hard to get. In contrast, I think that SG1 should own maintaining a representation of what SG1 thinks. If we agree that what I

propose above would help use, then we need to set up rules and processes that prevent us from ignoring this prior understanding. For example, we could say that in the meeting, we don't advance to the next topic of discussion until we have at least a skeleton of the changes required to represent the discussion and a volunteer to finalize them; that's similar to the approach we use for minute-taking duties right now. I think that over time and with more practice, it will become more natural to maintain this (e.g., clearly tracking open questions during discussions can certainly help to keep discussions on track; all we additionally need to do is write it down).

I have started this for the Executors proposal, and sent out an initial file for review. I got some feedback that it would be helpful to understand what's going on regarding Executors. Unfortunately, I got very little in terms of reviews (i.e., positive or negative critique) so far. My experience from open-source projects indicates that organizing reviews is something the community (so SG1 in our case) has to own (e.g., by raising incentives for people to review proposals before meetings).

Improve the consensus-building processes

Reaching consensus is essential for us; for example, consensus to forward a paper or consensus on a certain design decision like enabling the use of Executors on control structures. We can't just look at ISO processes for that because we make a lot of decisions before it comes to something that ISO cares about (e.g., consensus on what open questions need to be addressed before we can vote on a proposal). Therefore, we need to have good processes for consensus.

The only process we currently have, basically, are straw polls. But we rarely agree upon open questions, in particular what the open questions are exactly, nor do we track which need to be answered for a particular proposal (though we're getting a bit better in that regard).

We never do straw polls between meetings. Hence, there is no consensus building between meetings, and this slows us down. We have email discussions between meetings, but it's a good process for consensus that allows one to at least somewhat objectively determine when consensus has been reached in a discussion.

We have no real systematic process for how we track blockers, dependencies, and priorities, both with regard to individual papers as well as SG1's schedule overall. We make point decisions in that space (i.e., straw polls), but figuring out the open questions for a particular paper is hard, for example. The SG1 chair does a good job of trying to keep us on track, but this should be really something that SG1 owns as a group, and to which everyone in SG1 contributes systematically.

I also believe that we don't sufficiently manage the resource we have (in particular, time) so that we can maximize consensus (e.g., remove features from a proposal if they slow down consensus significantly, and if we can add them easily later on). We have some discussion around that every now and then, but lack of a process and consensus also means a lack of clarity of feedback to authors of proposals.

Meetings shouldn't be a bottleneck

Focusing on meetings as primary occasions to do work is a bottleneck. Of course, we prepare revisions of papers between meetings and there are some email discussions, but meetings are still where we make most progress such as reaching consensus on something.

Thus, I think we should try to do more work between meetings. This is **not** about doing more work in total, but about removing constraints that require certain things to happen at the meetings. Such constraints are bad for efficiency because it creates a more constrained schedule, and is more likely to cause conflicts with the schedules of individual contributors. Meetings should be where we discuss the hard topics that we couldn't reach consensus on previously, not where we take care of all the small stuff we could do over email.

There is little continuous integration, except at meetings where we then try to catch up quickly. Note that integration in our case doesn't even necessarily mean reaching consensus between meetings; it would already be

helpful if, for example, we could go over a new revision of a paper and mark which open questions from the previous meeting have been addressed or not, so that this isn't something we have to do in the full group at the meeting. Also, authors of a proposal often retreat to work just among themselves between meetings. This limits the possible feedback to the frequency of meetings; enabling others to give feedback early can make this more efficient.

It would be good if we would have a process for "pre-deciding" between meetings, with official approval of those earlier tentative decisions during meetings (unless the opinions have changed and there is no consensus at the meeting, of course). For example, I believe a lot of wording issues can often be solved more efficiently by asking experts between meetings than by discussing in full group in SG1 during meetings. A precondition for doing that is of course that we better track the work we have to do (and do so between meetings).

We don't give a lot of guidance to proposal authors regarding how they should contribute (beyond the act of writing a paper). We don't say a lot about how to address feedback, for example; I think we should request proposal authors to proactively track open questions and feedback raised for their papers (in some balance with SG1 as a whole).

Future work: Reaching out beyond SG1 to all of the community

While I think that we should first focus on improvements that are effective if applied just in SG1, I also think that reaching out beyond SG1's core and contributors would make SG1's core more efficient. In a nutshell, we need participation across the whole community: We work for the users, but we also depend on the users to give useful feedback to TSeS; without good feedback, the TS process will not work.

We have quite a few obstacles to participation across the four groups of the community, which decreases efficiency for us. For example:

- It's hard for followers to follow the steps the committee takes between specifications that are produced. IOW, SG1's collective brain is inaccessible between publications, and at publication time it's only the specifications that are easily visible.
- If followers can't follow along in a timely manner (eg, know what will likely be in a TS before this TS is published), then they can't prepare for closer evaluation of the specifications SG1 produces. This is bad because it increases the time until we get valuable feedback. In turn, this is critical for our overall progress.
- The same applies for users. They also can't be taught how to use the specifications we approach by followers because it's hard for the followers to prepare such educational content (or to just understand the design for themselves) before a specification is published.
- Papers are public, but the discussions of the papers are not. This is like giving people access to submitted patches but making patch review private.

A lot of these issues can be addressed by (1) implementing the improvements I suggested previously and then (2) making this accessible in some way and form to people outside of the committee. This just widens what we understand to be our community of active contributors, while the underlying open source principles remain the same.